

1. Write a python programme to solve 8 puzzle problem

Aim:

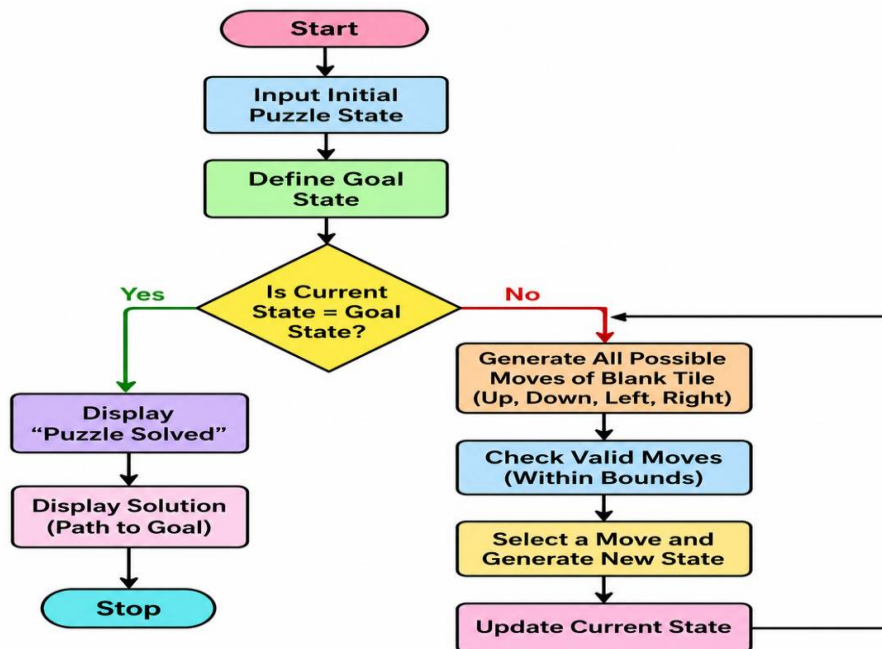
Write a program to solve 8 puzzle problem.

Objectives: The 8-puzzle problem is a game with 8 numbered tiles and 1 blank space (0) arranged in a 3×3 grid. The objective is to move the blank space up, down, left, or right to arrange the tiles in the correct order and reach the goal state.

Algorithm:

1. Start.
2. Define the initial puzzle state.
3. Define the goal state.
4. Compare the current state with the goal state.
5. If they are the same, display "Puzzle Solved".
6. Otherwise, move the blank tile (0) in a valid direction.
7. Repeat steps 4–6 until the goal state is reached.
8. Stop.

Flowchart:



Source Code:

```
from collections import deque
```

```
goal = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))
```

```
start = ((1, 2, 3),
         (4, 0, 6),
         (7, 5, 8))
```

```
def get_next_states(state):
    states = []
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j
```

```
    moves = [(-1,0), (1,0), (0,-1), (0,1)]
```

```
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new = [list(row) for row in state]
            new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
            states.append(tuple(tuple(row) for row in new))
```

```

return states

def solve(start):
    queue = deque([start])
    visited = set()

    while queue:
        state = queue.popleft()

        if state == goal:
            print("Puzzle Solved!")
            for row in state:
                print(row)
            return

        visited.add(state)

        for next_state in get_next_states(state):
            if next_state not in visited:
                queue.append(next_state)

    print("No Solution Found")

```

solve(start)

Result:

The program was executed successfully. The blank tile (0) was moved correctly, and the initial puzzle state was transformed into the goal state. Hence, the 8-puzzle problem was solved successfully.

Output:

Programiz
Python Online
Compiler

Programiz
PRO

main.py

Run

Output

Clear

```
1 from collections import deque
2
3 goal = ((1, 2, 3),
4         (4, 5, 6),
5         (7, 8, 0))
6
7 start = ((1, 2, 3),
8         (4, 0, 6),
9         (7, 5, 8))
10
11 def get_next_states(state):
12     states = []
13
14     for i in range(3):
15         for j in range(3):
16             if state[i][j] == 0:
17                 x, y = i, j
18
19                 moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
20
21                 for dx, dy in moves:
22                     nx, ny = x + dx, y + dy
23                     if 0 <= nx < 3 and 0 <= ny < 3:
24                         new_state = [list(row) for row in
25                                     state]
26                         new_state[x][y], new_state[nx][ny] =
27                             new_state[nx][ny],
28                             new_state[x][y]
29                         states.append(tuple(tuple(row) for
30                                             row in new_state))
31
32     return states
33
34 def bfs(start):
35     queue = deque([(start, [])])
36     visited = set()
37
38     while queue:
39         state, path = queue.popleft()
40
41         if state == goal:
42             return path + [state]
43
44         if state in visited:
45             continue
46         visited.add(state)
47
48         for next_state in get_next_states(state):
49             queue.append((next_state, path +
50                           [state]))
51
52     return None
53
54 solution = bfs(start)
55
56 if solution:
57     print("Solution found:")
58     for step in solution:
59         for row in step:
60             print(row)
61 else:
62     print("No solution found.")
```

Solution found:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

=== Code Execution Successful ===